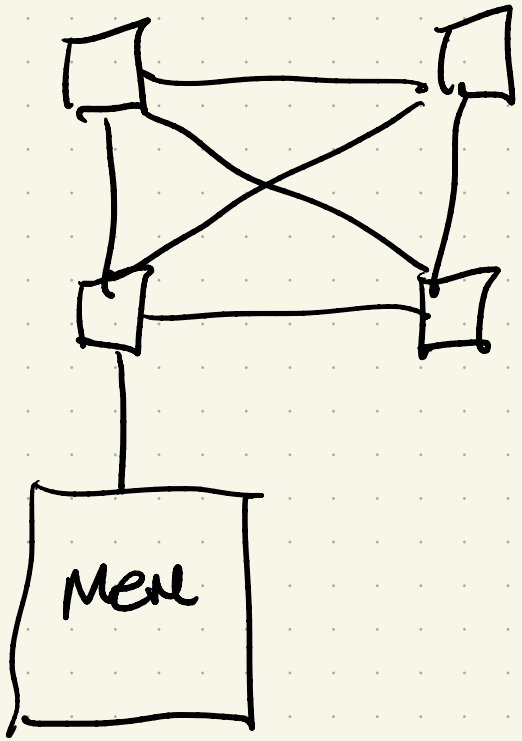
Snooping cache coherence

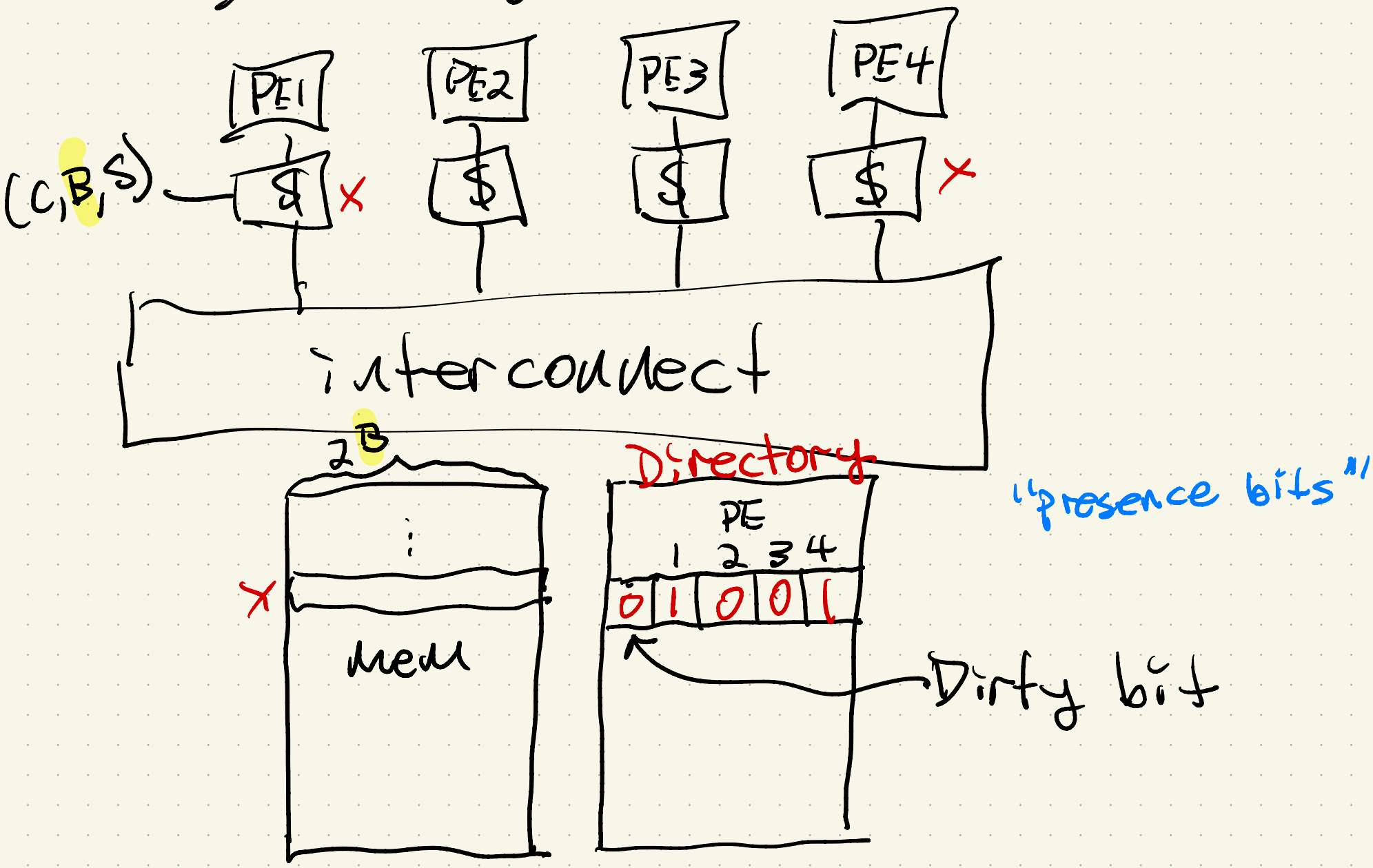
MOESIF

Requires an observable communications medium (eg. shared bus)

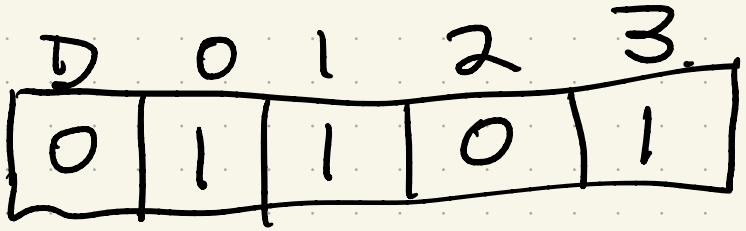
What about point-to-point?



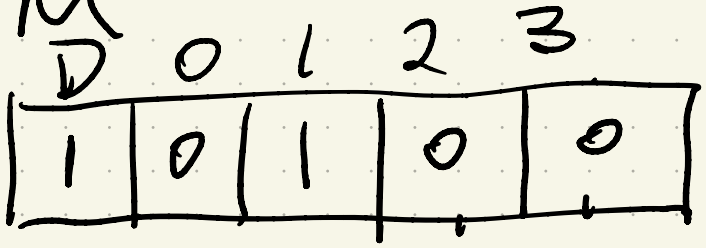
Censier and Feautrier in 1978
Directory Based Coherence



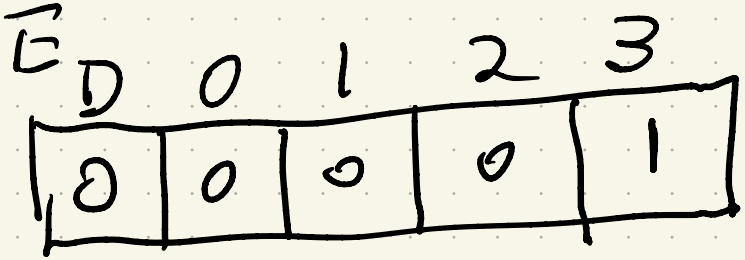
P0, P1 and P3 in S : P2 is in I



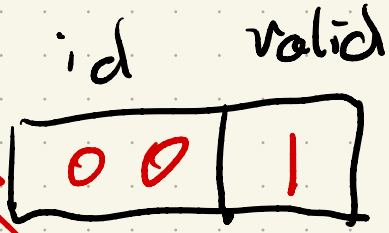
P1 in M



P3 in E

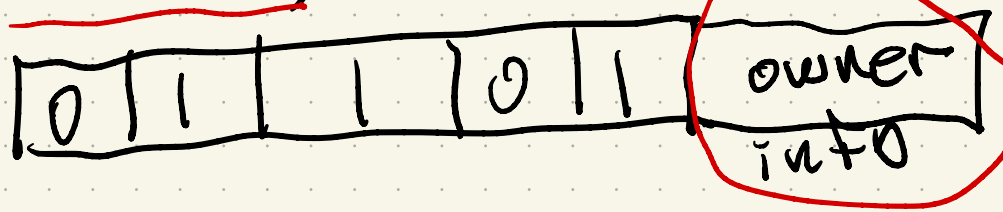


Add :



O or F state?

P0 in O, P1 and P3 in S

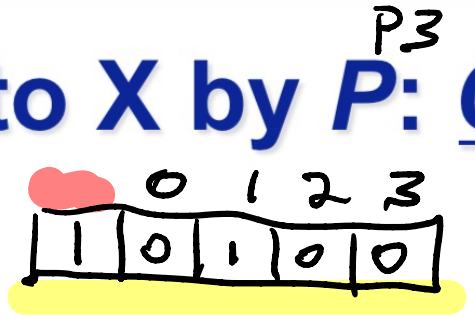


Messages for MSI for C&F

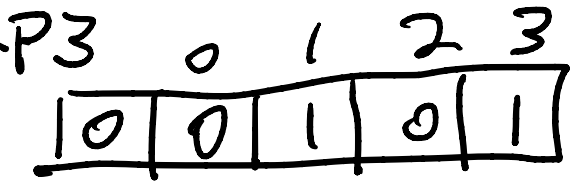
Messages from directory to proc	***Meaning***
recall_goto_S(P, X)	P must send the directory the data for block X and then P's X goes to S.
recall_and_invalidate(P, X)	P must sends the directory the data for block X and P's X goes to I
invalidate(P, X)	P invalidates its data, sends ACK to directory
send(P, X)	P receives data X
sendACK(P, X)	P receives an ACK from dir. for X

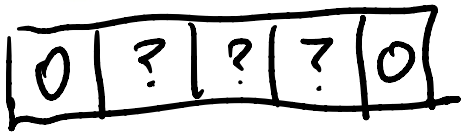
Messages from proc to directory	***Meaning***
GetM(directory, X)	Get X from directory with intent to modify, Proc waits for ACK
GetS(directory, X)	Get X from memory read-only
Ack(dir, X)	Send directory an ACK for X

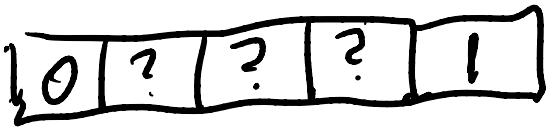
MSI: Read Miss to X by *P*: *GetS(dir, X)*



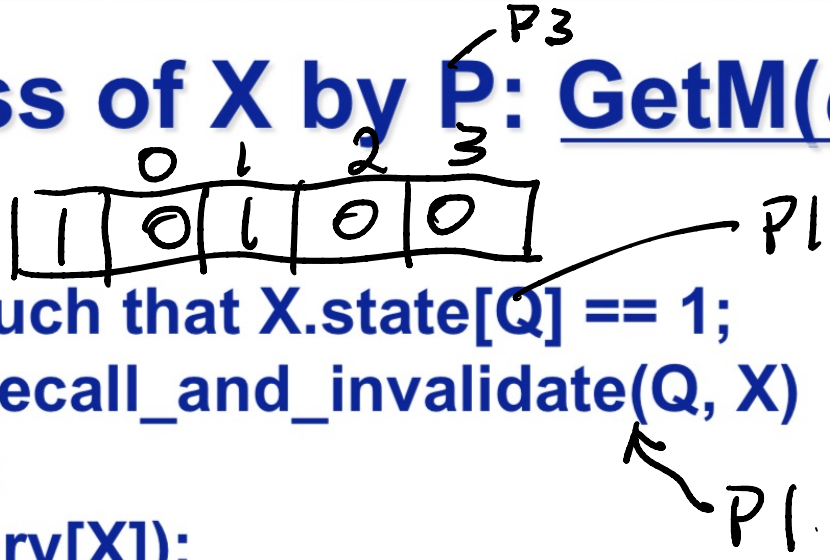

```
if (X.state.dirty) {
        // Assume Q such that X.state[Q] == 1;
        memory[X] = recall_goto_S(Q, X);
        X.state[Q] = 1;        // not really needed
        send(P, memory[X]);
        X.state[P] = 1;
        X.state.dirty = 0;
    } else {
        X.state[P] = 1;
        send(P, memory[X]);
    }
```







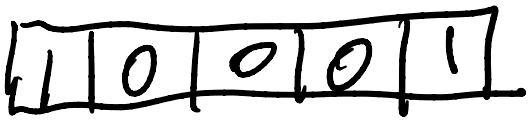
MSI: Write Miss of X by P: GetM(*dir, X*)



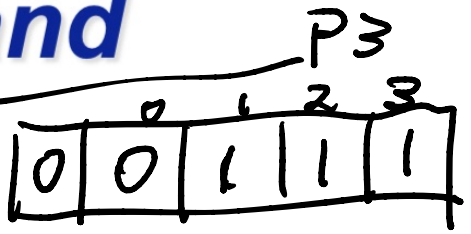

```
if (X.state.dirty) {
        // Assume Q such that X.state[Q] == 1;
        memory[X] = recall_and_invalidate(Q, X)
        X.state[Q] = 0;
        send(P, memory[X]);
        X.state[P] = 1;
} else {
        for each Q where X.state[Q] == 1 do {
                invalidate(Q, X)
                X.state[Q] = 0; }
        wait for all sharers to send ack messages
        send(P, memory[X]); // Note also sends ACK to P
        X.state[P] = 1;
        X.state.dirty = 1;
}
```







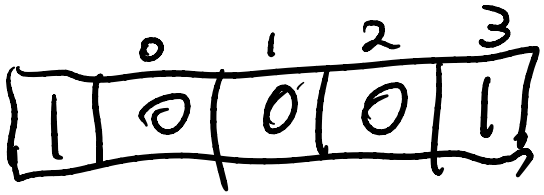
MSI: Write Hit to X for *P and* X is in S*: GetM(dir, P)*



P1 and P2

```
for each Q != P where X.state[Q] == 1 do {
        invalidate(Q, X)
        X.state[Q] = 0; }
// wait for all sharers to send ack messages
sendACK(P, X);
X.state[P] = 1;
X.state.dirty = 1;
```

P3 "it's OK to write block"



Key directory coherence problem

Size of directory!

Dir size ~ Memory size

Each directory entry is:

$(P + of_state_info + D)$

where $of_state_info = \lceil \lg P \rceil + 1$

where $D = 1$ if M or O state

total overhead is

$$(P + of_state_info + D) \times \frac{\text{size of mem}}{2^B}$$

Limited pointer schemes

i potential sharers

Dir-i-B

$i \times \lceil \lg P \rceil$ bits $P = 32$, $i = 2$

$2 \times 5 = 10$ bits

valid bit, dirty bit, broadcast bit

if > i GetS requests then

set broadcast bit

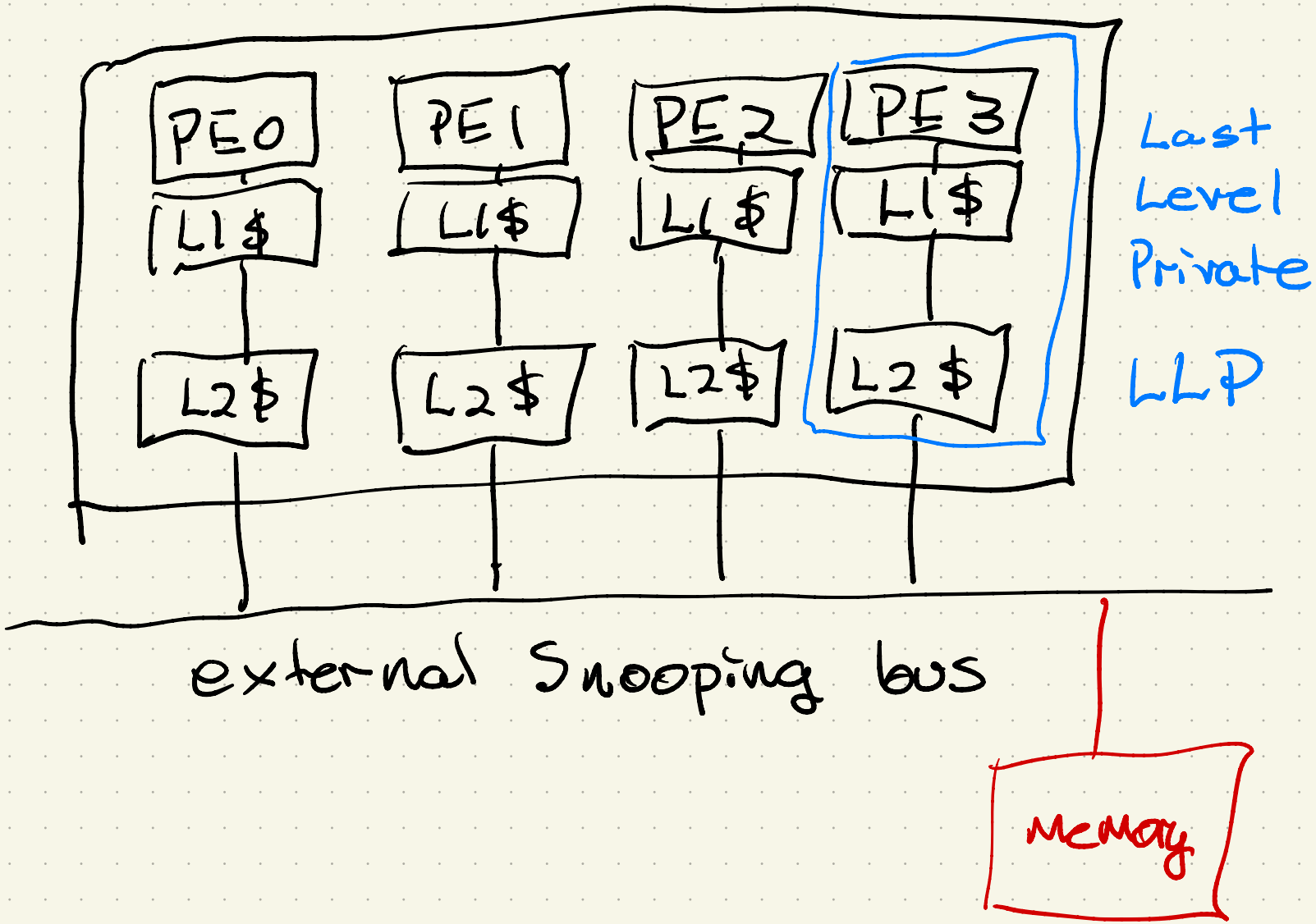
if GetM req and need to invalidate

then if broadcast bit == 1 then

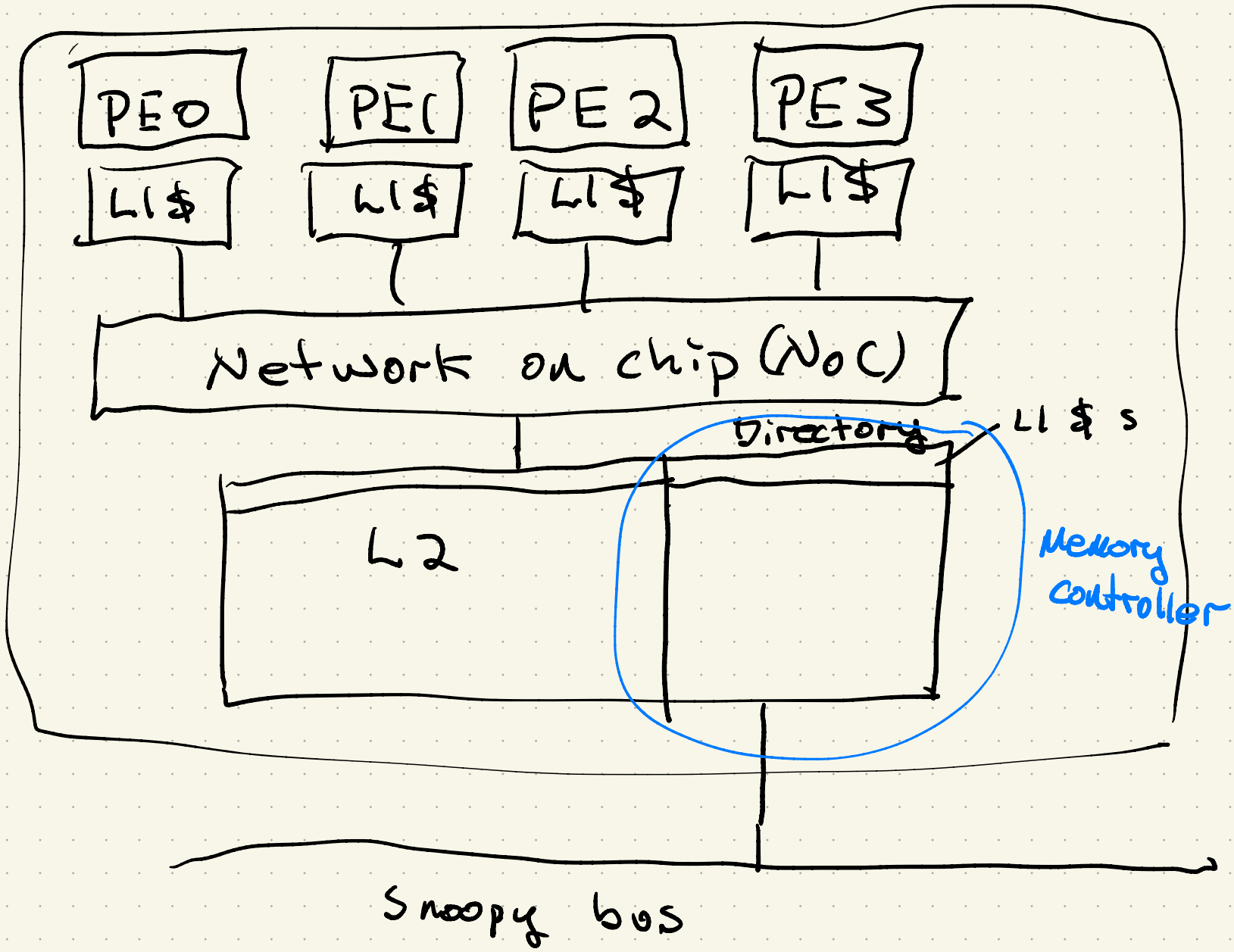
send all caches invalidate

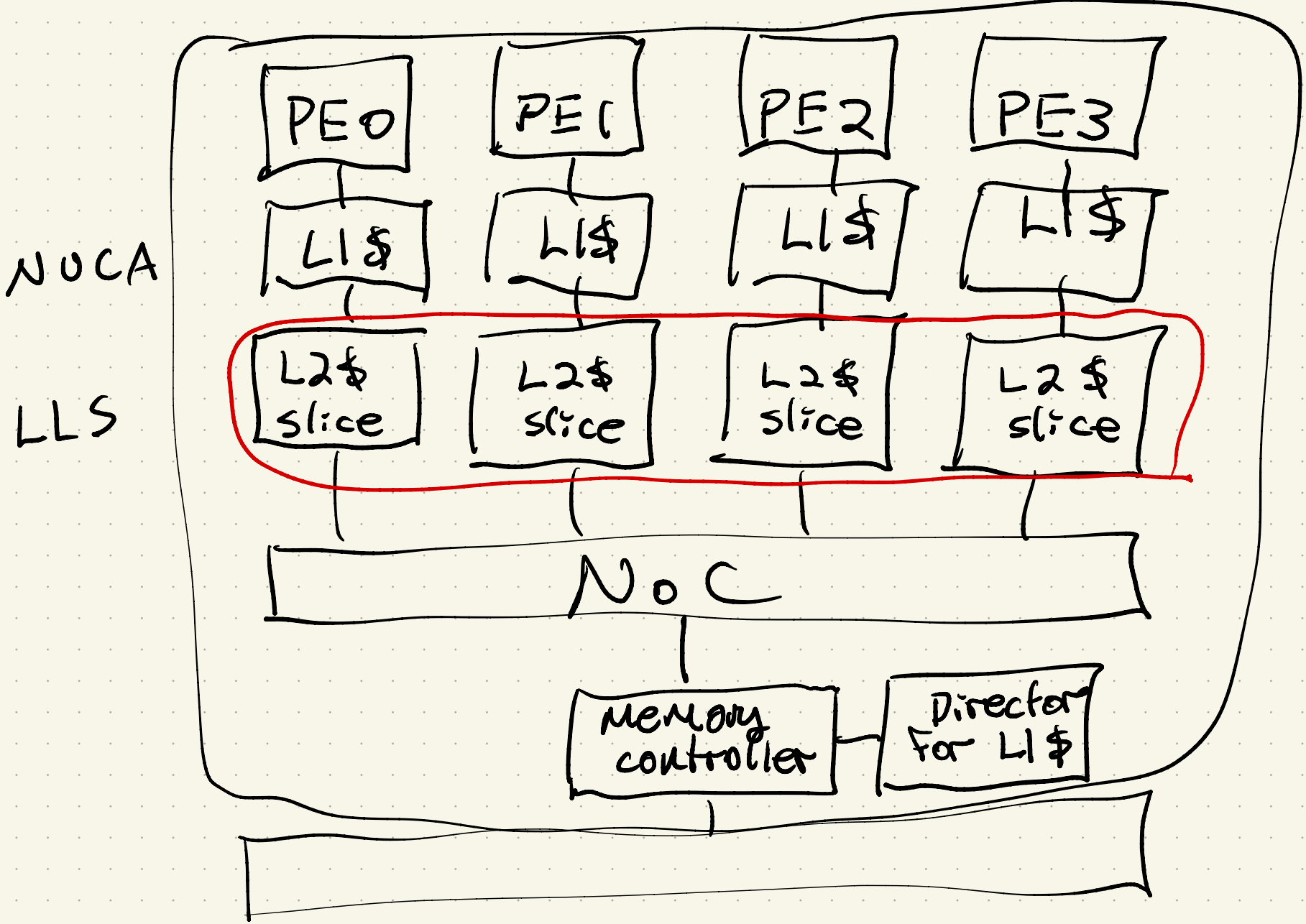
Dir-i-NB; if > i GetS, randomly

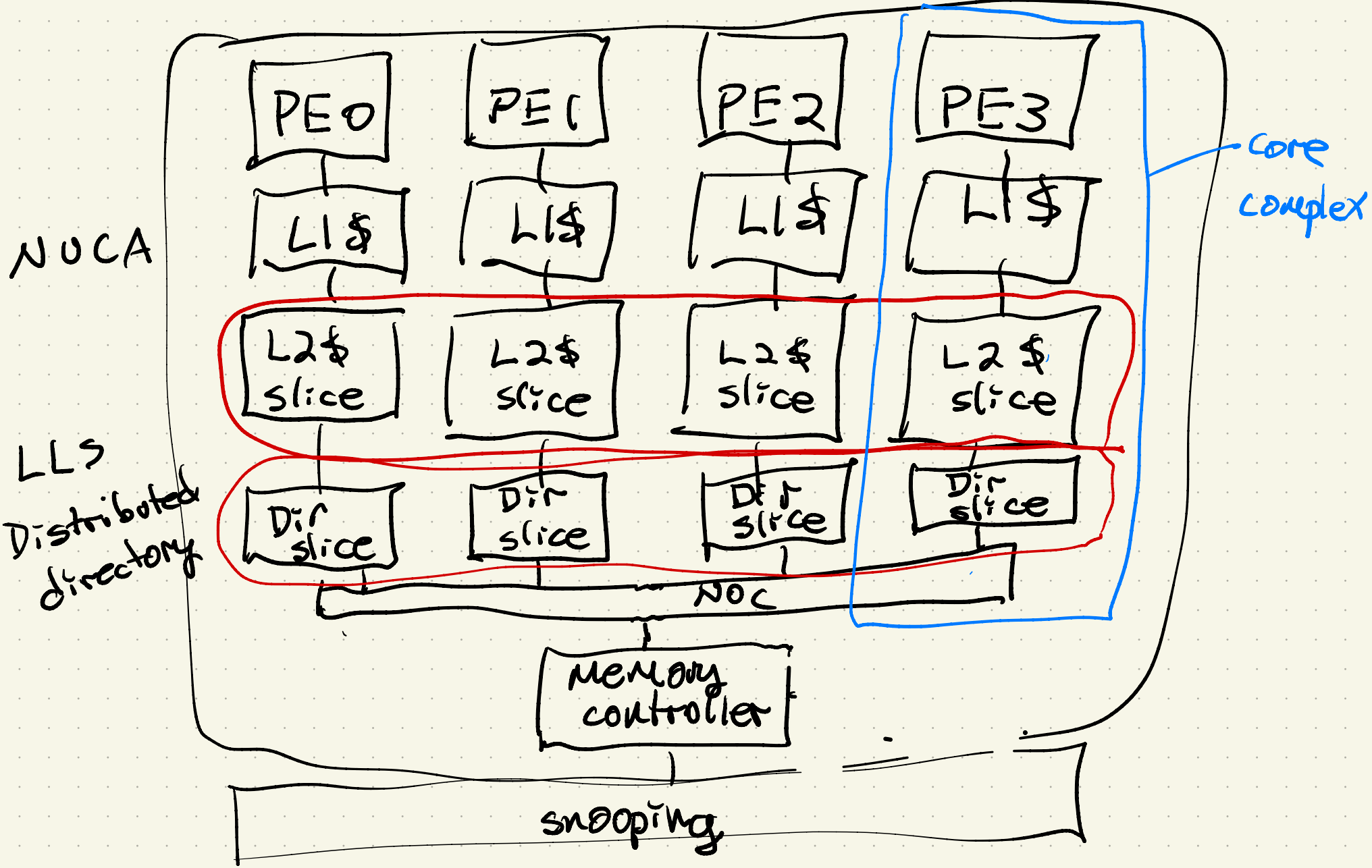
invalidate a sharer


PE0
PE1
PE2
PE3
L1$
L1$
L1$
L1$
L2$
L2$
L2$
L2$
Last Level Private
LLP
external Snooping bus
Memory

Last Level Shared




NUCA
LLS
PE0
PE1
PE2
PE3
L1$
L1$
L1$
L1$
L2$ slice
L2$ slice
L2$ slice
L2$ slice
NoC
Memory controller
Director for L1$


PE0
PE1
PE2
PE3
L1$
L1$
L1$
L1$
L2$ slice
L2$ slice
L2$ slice
L2$ slice
Dir slice
Dir slice
Dir slice
Dir slice
NoC
Memory controller
snooping
core complex
NUCA
LLS
Distributed directory